



EI-LSH: An early-termination driven I/O efficient incremental c -approximate nearest neighbor search

Wanqi Liu^{1,2} · Hanchen Wang^{1,2} · Ying Zhang² · Wei Wang³ · Lu Qin² · Xuemin Lin³

Received: 21 November 2019 / Revised: 6 August 2020 / Accepted: 1 September 2020 / Published online: 30 September 2020
© Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract

Nearest neighbor in high-dimensional space has been widely used in various fields such as databases, data mining and machine learning. The problem has been well solved in low-dimensional space. However, when it comes to high-dimensional space, due to the curse of dimensionality, the problem is challenging. As a trade-off between accuracy and efficiency, c -approximate nearest neighbor (c -ANN) is considered instead of an exact NN search in high-dimensional space. A variety of c -ANN algorithms have been proposed, one of the important schemes for the c -ANN problem is called Locality-sensitive hashing (LSH), which projects a high-dimensional dataset into a low-dimensional dataset and can return a c -ANN with a constant probability. In this paper, we propose a new aggressive early-termination (ET) condition which stops the algorithm with LSH scheme earlier under the same theoretical guarantee, leading to a smaller I/O cost and less running time. Unlike the “conservative” early termination conditions used in previous studies, we propose an “aggressive” early termination condition which can stop much earlier. Though it is not absolutely safe and may result in the probability of failure, we can still devise more efficient algorithms under the same theoretical guarantee by carefully considering the failure probabilities brought by LSH scheme and early termination. Furthermore, we also introduce an incremental searching strategy. Unlike the previous LSH methods, which expand the bucket width in an exponential way, we employ a more natural search strategy to incrementally access the hash values of the objects. We also provide a rigorous theoretical analysis to underpin our incremental search strategy and the new early termination technique. Our comprehensive experiment results show that, compared with the state-of-the-art I/O efficient c -ANN techniques, our proposed algorithm, namely EI-LSH, can achieve much better I/O efficiency under the same theoretical guarantee.

Keywords Locality-sensitive hashing · Approximate nearest neighbor search · Similarity search · I/O efficient algorithm

1 Introduction

Given a set of d -dimensional objects (points) and a query object (point), nearest neighbor (NN) search finds the object which has the smallest distance to the query object. This is a fundamental and essential operation in the applications of many domains, including databases, computer vision, multimedia, machine learning and recommendation systems. In these applications, each object is usually represented by a point with high dimensionality. For instance, by utilizing the deep learning technique [22], an image can be embedded into a point in a 4096-dimensional space. Then, for a given image (i.e., a high-dimensional point), an NN search can be used to identify the most similar one within an image database. Due to the prevalence of graph embedding techniques (e.g., DeepWalk [34] and node2vec [14]), the graph nodes (e.g., users, products and entities) in many applications such as e-

✉ Hanchen Wang
hanchenw.au@gmail.com
Wanqi Liu
wanqi.liu@student.uts.edu.au
Ying Zhang
ying.zhang@uts.edu.au
Wei Wang
weiw@cse.unsw.edu.au
Lu Qin
lu.qin@uts.edu.au
Xuemin Lin
lxue@cse.unsw.edu.au

¹ Zhejiang Gongshang University, Hangzhou, China
² AAIL, University of Technology, Sydney, Australia
³ The University of New South Wales, Sydney, Australia

commercial networks and knowledge graphs are embedded as point objects (vectors) in a high-dimensional space based on their topology structure in the graph. The NN search has been intensively used on these embedded objects for a variety of purposes such as recommendation and near-duplicate user detection.

The NN search problem has been well solved when the dimension is a small value, but when it comes to high-dimensional space, due to the “curse of dimensionality,” it becomes very expensive to find the exact NN point and difficult to distinguish between similar objects and measure their distances meaningfully. Nevertheless, returning sufficiently nearby objects, referred to as an Approximate Nearest Neighbor Search (ANNS), can be undertaken efficiently and is sufficiently useful for many practical problems, thus attracting an enormous number of research efforts. There are two categories of approximate NN search algorithms: (1) c -approximate NN (c -ANN) search algorithms, which have a (c, δ) -approximate theoretical guarantee; and (2) approximate NNS algorithms without such a theoretical guarantee. Both categories have attracted much research attention in recent years, but our research focus is on the c -ANN search. Specifically, given an approximate ratio c ($c > 1$) and a success probability δ ($0 < \delta < 1$), a c -ANN search algorithm will return a c -approximate solution with a confidence of at least δ . As stressed in [24], in addition to the promising theoretical results, c -ANN algorithms have more robust performance against hard query loads and their indexes are very friendly to updates.

In many applications, the volume of data is huge due to the large number of objects and high dimensionality. For instance, as reported in [40], the billion-scale objects (around 1 billion users and 2 billion items) are embedded as 160-dimensional point objects at Alibaba for user recommendation. To handle large-scale data which cannot fit in the main memory, there are two popular and practical solutions: an external memory-based solution [6,45] and a distributed solution [2,16]. In this paper, we develop a new early-termination condition for the hash table-based LSH algorithms in external memories which can stop the algorithm earlier without breaking the theoretical guarantee. Furthermore, we also adopt an increasing searching strategy to reduce the I/O cost and to support the early-termination condition.

1.1 Motivation

Locality-sensitive hashing (LSH) [19] is a widely adopted method to support the c -ANN search. In addition to a theoretical guarantee, it also enjoys great success in practice due to its excellent performance and ease of implementation. The recent three I/O efficient c -ANN algorithms for Euclidean distance, C2LSH [11], SRS [37] and QALSH [18],

are variants of LSH and a set of m 2-stable hash functions is employed for the random projection of high-dimensional data. In particular, each object in d -dimensional space will be hashed into m real values by these m hash functions where $m \ll d$. These hashed values are organized in two ways: (1) a multi-dimensional index (e.g., R-tree) is deployed to keep the m hash values for each object (e.g., SRS), which means there is only one data entry for each object; and (2) m B+ trees are employed to separately keep the m hash values of each object (e.g., C2LSH and QALSH), and there are m data entries for each object.

In SRS [37], an R-tree is used to index the objects in the m -dimensional projected space, where there is one data entry for each object (i.e., an m -dimensional point with object ID). The exact NN search is conducted on the R-tree for the c -ANN search. The most impressive property of SRS is the small index size compared to existing LSH-based methods including C2LSH and QALSH. With the approximate ratio $c = 4$ with the success rate $\delta = \frac{1}{2} - \frac{1}{e}$ on Mnist data [18,37], the number of hash functions m is set to 6, 65 and 115 for SRS, C2LSH and QALSH, respectively. The small number of hash functions enables SRS to fit the index in the main memory even for considerably large-sized data. However, SRS is not I/O efficient when the index has to reside in the external memory. In practice, the existing multi-dimensional indexes (e.g., R-tree) cannot support the exact NNS on data with a projected dimension (i.e., number of hash functions m) larger than 5, and the performance drops dramatically with a growth in the dimensionality. This is because of the loss of the local proximity of objects on the same page with the growth of dimensionality and random I/O costs invoked.

Incremental search In C2LSH and QALSH, each object has m entries in m B+ trees. An advantage of B+ tree is that the sequential I/O can be used if we pre-fetch a set of sorted records. The main issue of the C2LSH and QALSH is that their search strategy may lead to a large number of I/Os. The key of the LSH is that we hope two close objects in high-dimensional space are also close to each other in each projected dimension. Assume there is a hash function such that the closeness of the objects to q are well preserved in the projected dimension. Intuitively, we should incrementally access the projected objects according to their distances to q in the projected dimension as shown in Fig. 1c. However, C2LSH and QALSH adopt the *bucket exponential expansion* strategy where the bucket widths must be a power of c (i.e., bucket widths grow exponentially), which may lead to some counter-intuitive scenarios. More specifically, QALSH set the bucket width to w initially, and at each round, the bucket width is enlarged by c times. Since at each round, QALSH and C2LSH both need to visit all points in the current bucket to keep their theoretical guarantee, but the c -ANN candidate may be found at the very beginning of this round, it is a waste of I/O and time to check all of the other points. We say an

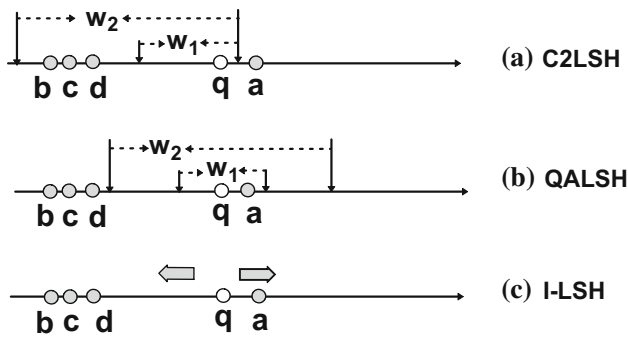


Fig. 1 Motivation for incremental searching

object collides with q if it is assigned to the same bucket with q in the projected dimension. The objects which collide with q will be explored, and the bucket width will keep growing if necessary (e.g., from w_1 to w_2 in Fig. 1a, b, and hence new objects may collide with q and be processed. It is shown in [18] that the LSH function in C2LSH is *query-oblivious*; that is, the possible boundary of the buckets has been pre-determined before the arrival of the query. In Fig. 1a, although the object a is close to q in the projected dimension, it will not be explored even if the bucket width grows from w_1 to w_2 . QALSH addresses this issue by proposing the *query-aware* LSH function where, as shown in Fig. 1b, the query is always centered at its corresponding bucket. By doing this, object a in the example will be explored at the beginning. Nevertheless, because the distribution of the hash values is not uniformly distributed in practice, the search may encounter too many new objects (e.g., Fig. 1a) or no new object (e.g., Fig. 1b) when bucket width w_1 is extended to w_2 .

Early termination condition In addition to the searching strategy, we observe that another possible improvement of the current list-based LSH scheme is the early-termination (ET) condition. In the list-based LSH scheme, there are generally two termination conditions: (1) if enough candidates are found, the algorithm returns the nearest candidate as the result, or (2) if a point o satisfies $\|o, q\| \leq cR$ then the algorithm returns o as the result, where q is the query point and R is the current radius of $B(q, R)$. The termination condition 2 is also called the early-termination condition. It is obvious that if the algorithm is terminated by the early-termination condition, the result will be a correct c -ANN according to the definition of this problem. There has been rigorous proof that termination condition 1 (the normal termination condition) can return a c -ANN with a constant probability in the worst case. Since there is only one false probability, the total success probability of LSH is easy to control, but the theoretical bound might be loose. In this paper, we propose a more aggressive early termination condition such that the algorithm can stop earlier and cost less I/O. In this case, another

false probability which is caused by the new ET will be considered. In the querying procedure, the projection distance of each object is easy to obtain from the index. Based on the p -stable distribution property, for a point o_{min} with a projection distance d to the query point, if we can compute the probability that there does not exist any point o which satisfies (1) $\|o, q\| \leq \|o_{min}, q\|/c$ and (2) o hasn't been found as a candidate, then set a lower bound p_{ET} of the probability. When a point o satisfied $p(o) \geq p_{ET}$ we can return the point as the result because it already has a large enough probability to be a correct answer. As long as we choose a suitable value for p_{ET} , the final success probability can be guaranteed to be at least δ . Thus, it is more challenging to achieve a theoretical guarantee due to the co-existence of two non-independent failure conditions. Nevertheless, we show that this will be well paid-off since it turns out that the use of the new ET condition can significantly improve the performance under the same theoretical guarantee.

In this paper, we follow the framework of C2LSH and QALSH to enjoy the high efficiency sequential I/O brought by the B+ tree. In addition, we provide a rigorous analysis to show that we can conduct the incremental search strategy as shown in Fig. 1c on each projected dimension and identify the c -ANN object with the desired confidence. By doing this, we show in Sect. 5 that our proposed algorithm, namely EI-LSH, can significantly improve the performance by: (1) deriving a more promising early termination condition; and (2) using approximate ratio c in our search algorithm, instead of $\sqrt{c}$ used in C2LSH and QALSH, due to the use of an incremental search strategy. Our experiments on real-life datasets demonstrate that EI-LSH can achieve the same theoretical guarantee with much lower I/O costs.

1.2 Challenges

The primary challenge in the problem is that it is difficult to calculate probability γ , which is the probability that the algorithm is terminated by the early-termination condition. We can set it to 50% to have a loose bound, but the optimal solution to achieve the best I/O performance requires a more accurate proportion. Because the new ET framework is dependent on the current minimum distance and the current projection radius, it is difficult to compute its exact probability of occurrence before the normal termination condition.

1.3 Contributions

Our principal contributions are summarized as follows.

- We propose a new c -approximate nearest neighbor search algorithm, namely EI-LSH, for high-dimensional data, which uses an incremental search strategy on the projected dimensions.

- We design a new early termination technique which can be used by most list-based LSH algorithms to aggressively reduce the number of objects accessed without breaking the theoretical guarantee.
- We provide a rigorous analysis to demonstrate the correctness and efficiency of our proposed methods.
- We perform an extensive performance evaluation against two state-of-the-art I/O efficient c -ANN algorithms regarding I/O costs and result accuracy. The results demonstrate that our proposed methods can achieve the best I/O performance under the same theoretical guarantee.

Contributions compared to conference version Compared to the I-LSH proposed in our conference version [25], we propose a novel aggressive early termination condition in this paper. The traditional early termination condition used in I-LSH stops the search process when there is a point o that satisfies $\|o, q\| \leq cr/w$ where r means the current bucket width and w denotes the initial bucket width. The aggressive early termination condition will stop the algorithm when there is a point o satisfies that $\|o, q\| \leq \lambda r$ where r is the current projection distance between o and q , and λ is a pre-computed parameter. Compared to I-LSH, EI-LSH could stop the search process much earlier while keeping the same theoretical guarantee. We also provide totally new analysis, which is more complicated than conference version since two failure conditions are considered in the analysis. We also provide theoretical analysis that guides the setting of the parameters to achieve better performance under the same theoretical guarantee. Furthermore, compared to the conference version, the comprehensive experiments are conducted on the larger-size datasets with more evaluation metrics.

Roadmap The rest of the paper is organized as follows. We formally define the problem and introduce the relevant work in Sect. 3. We then present our incremental search algorithm with new early termination techniques in Sect. 4. The theoretical analysis and comprehensive experiments are detailed in Sects. 5 and 6, respectively. Finally, Sect. 7 concludes the paper.

2 Related work

In this section, we introduce the relevant existing works.

2.1 List-based LSH algorithm

A major solution to overcome the multiple indexes of LSH is the virtual rehashing, which has been used in some LSH papers with a theoretical guarantee such as LSB-tree [38], C2LSH [11] and QALSH [18].

Basically, instead of physically constructing the buckets with different widths to cope with the possible radius (i.e., R values) in c -NN search, the virtual rehashing increases the bucket width based on one hash function only (i.e., random projection of the objects). Specifically, suppose the original hash function $H(o)$ is $(1, c, p_1, p_2)$ -sensitive for $R = 1$. To enlarge the search radius from 1 to c , we may simply change the hash function to $H^c(o) = \frac{H(o)}{c}$, where $H^c(o)$ is (c, c^2, p_1, p_2) -sensitive. Then, the above procedure is repeated until the termination of the algorithm (i.e., retrieves enough candidates or meets the early termination condition).

Virtual rehashing methods can be grouped into two categories: query-oblivious LSH and query-aware LSH. In the following, we introduce two representative works, C2LSH [11] and QALSH [18], in each category, respectively. C2LSH [11] uses the virtual rehashing function to increase the width of the bucket. Instead of following the AND-then-OR scheme [6], C2LSH does not use the LSH table containing a set of hash functions (i.e., AND scheme) but uses a set of m LSH functions. For an object o , if o collides with query q (i.e., in the same bucket) in at least αm LSH functions, then o will be added to the candidate set. The algorithm will terminate if there are already βn candidates during the search or there exists an object o such that $\|o, q\| \leq cR$ where R is the current search radius. The advantage of C2LSH is that it does not require a complex hash table, which can significantly reduce the total number of hash functions. As a result, C2LSH has a much smaller I/O cost compared with LSH and LSH-forest. One disadvantage of C2LSH is that its LSH function is *query-oblivious*; that is, the possible virtual partitions have been determined before the arrival of the query. As illustrated in Fig. 1a, an object close to q in the projected space may be assigned to different bucket even if we keep on increasing the width of the bucket. LSB-Tree [38] uses a set of LSB-tree to build an LSB-forest instead of hash tables. In each LSH-tree, a data object is projected to a value where $h(o) = \mathbf{a} \cdot \mathbf{o} + b^*$. The vector $\mathbf{a}$ and $\mathbf{o}$ have the same meaning as other LSH methods, and b^* is uniformly distributed in $[0, 2^f w)$ where f is a variable related to the dimensionality of the largest coordinate in each dimension, and w is a constant. LSB-tree uses a Z-order technique to implement the virtual rehashing step. Each data object o is converted to an m -dimensional data object $G(o) = (h_1(o), \dots, h_m(o))$ where $h_i(\cdot)$ is the i_{th} hash function, and then, each $G(o)$ is converted to the Z-order value $z(o)$. The Z-order values are used to build the LSB-tree. The basic idea of an LSB-tree is to evaluate the “closeness” between two Z-order values using the notion of Length of the Longest Common Prefix (LLCP). The Z-order values stored in the leaf nodes will be visited in a decreasing order of the LLCP with $z(q)$, which simulates the process of processing a series of (R, c) -NN search with increasing radius R .

LSB-tree generally has excellent performance in terms of result accuracy, but the I/O cost is much higher than other algorithms. Furthermore, the LSH function is also *query-oblivious*. Both the LSB-tree and C2LSH have a major defect in that they require c to be a power of 2, and since the approximation ratio of LSB-tree and C2LSH is c^2 , the best guarantee they can keep is a 4-approximate nearest neighbor, which might not be accurate enough in some scenario.

QALSH [18] addresses the above issue by proposing the *query-aware* virtual rehashing function. As shown in Fig. 1, *QALSH* always sets the query point as the center of the bucket such that the point close to q in the projected space will have a good chance to share the same bucket. Particularly, the hash function of *QALSH* is defined as: $h(o) = \mathbf{a} \cdot \mathbf{o}$, and for each hash function, the hash values are stored in a $B+$ tree. When q arrives, the query point will always be at the center of the *anchored bucket*, and the data object whose hash value $h(o)$ satisfies $|h(o) - h(q)| \leq \frac{w}{2}$ collides with q . Although *QALSH* can enhance the search performance with the *query-aware* technique and can accept any c greater than 1, it still has some drawbacks. *QALSH* has to iteratively increase the bucket width and access all objects touched by the expansion of the bucket. As shown in Fig. 1a, it may explore many unnecessary objects because it is difficult to find a good way to expand the bucket.

2.2 Tree-based LSH method

Although LSB-tree and *QALSH* use B+tree or B-tree to build the index, they are still considered as list-based LSH algorithms since the searching is done on single hash functions whose structures are lists, and the tree structure is simply used to speed up the search. A tree-based LSH method uses a multi-dimensional index. In a nutshell, SRS [37] reduces the problem of “approximate NN search in high dimensional space” to an “exact T-NN search in a low dimensional space”. As shown in [32,37], if we use m independent 2-stable function to map each data point in $\mathcal{R}^d$ to m -dimensional space $\mathcal{R}^m$, the square of L_2 distance between two objects o_1 and o_2 within $\mathcal{R}^m$ (namely projected distance) follows the χ^2 distribution with m degrees of freedom regarding the square of their distances in $\mathcal{R}^d$. SRS uses the classical multi-dimensional index, e.g., R-tree, to organize the objects in $\mathcal{R}^m$ space. Then, an incremental exact NN search can be conducted to solve the problem. The key idea of SRS is very intuitive and the index size is very small compared with all the other c -ANN methods under the same theoretical guarantee. *Discussion* The main limit of SRS is I/O efficiency when the index cannot be held in the main memory. According to the algorithms of SRS, the number of projected dimensions m increases quickly with regard to the approximate accuracy. Moreover, as shown in the empirical study, a considerable large number of objects (i.e., disk pages) are accessed during

the search. On the other hand, it is well known that a multi-dimensional index like R-tree cannot efficiently support an exact NN search for data with projected dimensionality over 5, and the performance drops dramatically when dimensionality increases due to the curse of dimensionality. When the approximation ratio c is set to be a smaller value, such as 2, SRS will require a 15-dimensional R-tree to undertake the exact NN search, but the performance will be extraordinarily bad. Moreover, I/O invoked in SRS is random I/O, which is much more expensive than sequential I/O.

2.3 Other relevant work

In this paper, we focus on c -ANN algorithms which have a rigorous theoretical guarantee. There are also some I/O efficient ANN search algorithms proposed in the literature without theoretical guarantee. For instance, Liu et al. [27] proposed SK-LSH for approximate NN problems as an improvement of LSB-tree, which used linear order instead of Z-order for better I/O efficiency. In [26], an I/O efficient algorithm was proposed based on the PQ method [20]. Arora et al. [1] proposed HD-Index which consisted of a hierarchical structure called RDB-tree to support the c -NN query in high-dimensional datasets. Gu et al. [15] applied the PCA [17] technique to project the data from high-dimensional space to low-dimensional space and proposed OR-tree to optimize the I/O cost.

In addition to the aforementioned external memory-based techniques, a large body of work has been proposed in the literature (e.g., Database, Machine Learning, Data Mining and Multimedia) to enhance the performance of approximate NNS assuming that the index and data can be held in the main memory without a theoretical guarantee. As shown in [24], they can be classified into four categories based on the nature of the techniques, namely: (1) *LSH-based methods* such as multi-prob LSH [28], Bi-level LSH [31]), and DSH [12]); (2) *Encoding-based methods* such as Spectral Hashing [43], Neighbor Sensitive Hashing [33], Selective Hashing [13], and Product Quantization [20], (see [41] for a comprehensive survey); (3) *Tree-based space partition* methods such as randomized kd-tree [36], FLANN [30] and Annoy [4]; and (4) *Neighborhood-based* methods such as KGraph [8], HNSW [29] and DPG [24]. It is also worth mentioning that some recent works take advantage of hardware to speed up the approximate NN search such as GPU [21,42] and FPGA [44].

3 Preliminaries

In this section, we present the problem definitions. Some important notations used throughout the paper are summarized in Table 1.

Table 1 Summary of notations

Notation	Definition
n	the number of point objects in the dataset
d	the dimensionality of the dataset
m	the number of hash functions (projected dimensions)
q	the query object
$\ o_1, o_2\ $	the Euclidean distance between o_1 and o_2
o^*, R^*	NN object of q , with distance R^*
o_{min}, R_{min}	c -ANN object returned, with distance R_{min}
$h_a(o)$	the hash value of point o using the vector $\mathbf{a}$
$d_i(o)$	projected distance w.r.t i -th hash function with $d_i(o) = h_i(o) - h_i(q) $
c and δ	approximate ratio and success probability
w	the initial bucket width in LSH functions
r	the search radius on projected dimensions
R	$R = \frac{2r}{w}$, radius of ball $B(q, R)$ in $\mathcal{R}^d$ space regarding search radius r
γ	the probability that the algorithm is terminated by the normal termination condition
R^+	$R^+ = \lambda r$, distance threshold for early termination where λ is a pre-computed constant

3.1 Problem definition

First we give the formal definition of the c -ANN problem. Considering a dataset D with n point objects in a d -dimensional space, denoted by $\mathcal{R}^d$. We are particularly interested in the high-dimensional case where d is a large number (e.g., $d \geq 50$). The coordinate value of an object o on the i_{th} dimension is denoted as $o[i]$. In this paper, we focus on Euclidean distance which is one of the most popular distance metrics widely used in a variety of applications. For a given query object q , the Euclidean distance between o and q is denoted by $\|o, q\| = \sqrt{\sum_{i=1}^d (o[i] - q[i])^2}$. For presentation simplicity, we use “distance of the object” to represent the “distance between the object and the query” whenever there is no ambiguity. The c -approximate nearest neighbor is defined as follows:

Definition 1 [c -approximate nearest neighbor (c -ANN)] For a given query object q and a d -dimensional dataset $\mathcal{D}$, suppose o^* is the nearest neighbor of q with distance R^* , a c -approximate nearest neighbor of q is a data object $o \in \mathcal{D}$ such that $\|o, q\| \leq cR^*$ where c is the approximate ratio.

Problem statement In this paper, we propose an efficient algorithm to solve the c -ANN problem in d -dimensional Euclidean space with a theoretical guarantee; that is, given the approximate ratio c and the probabilistic threshold δ , the algorithm should return the c -approximate nearest neighbor

(c -ANN) with probability at least δ . The theoretical guarantee will always hold regardless of the data distribution.

3.2 LSH with bucket partitioning

Locality-sensitive hashing (LSH) was first introduced by Indyk and Motwani [19] to solve the c -ANN problem in binary Hamming space and was extended by Datar et al. [6] to Euclidean space.

In a nutshell, the LSH functions map the objects in a d -dimensional dataset $\mathcal{D}$ into an m -dimensional space, denoted by $\mathcal{R}^m$, where $m \ll d$ and the relative distance among the objects are kept in the mapped space. That is, if two objects p_1 and p_2 are close to each other in d -dimensional space, hopefully they will be mapped to nearby positions after a random projection. By partitioning these real values (i.e., one dimensional data) with a width w , the objects in $\mathcal{R}^d$ are mapped to different *buckets*. Intuitively, two close objects in space $\mathcal{R}^d$ will have a good chance of being mapped into the same bucket, and vice versa for the two distant objects.

An LSH function family $\mathcal{H}$ for a distance function f is defined as (r_1, r_2, p_1, p_2) -sensitive if and only if for any two data objects x and y , there exists two distance thresholds r_1 and r_2 and two probability thresholds p_1 and p_2 that satisfy: $\begin{cases} \Pr_{H \in \mathcal{H}}[H(x) = H(y)] \geq p_1, & \text{if } f(x, y) < r_1 \\ \Pr_{H \in \mathcal{H}}[H(x) = H(y)] \leq p_2, & \text{if } f(x, y) > r_2 \end{cases}$. This implies that the chance of mapping two objects x, y to the same hash value (i.e., *bucket*) grows as their distance $f(x, y)$ decreases.

In [6], an LSH function H can be formally represented by $H_{\mathbf{a},b}(o) = \frac{\mathbf{a} \cdot \mathbf{o} + b}{w}$ where $\mathbf{a}$ is a d -dimensional vector for the random projection, o is the d -dimensional data object, w is the bucket width, and b is a variable randomly chosen from $[0, w]$. The p -stable distribution, e.g., the Gaussian/normal distribution for $p = 2$, and Cauchy distribution for $p = 1$, can be used to construct $\mathbf{a}$ of the LSH function for the random projection. A formal definition of p -stable distribution follows.

Definition 2 (p -stable distribution [6]) A distribution D over $\mathcal{R}$ is called p -stable, if there exists $p \geq 0$ such that for any n real numbers $v_1, \dots, v_n$ and i.i.d. variables $X_1, \dots, X_n$ with distributions D , the random variable $\sum_{i=1}^n v_i \cdot X_i$ has the same distribution as $(\sum_{i=1}^n v_i^p)^{1/p}$ where X is a random variable with the distribution D . p -stable distribution has only been found when $p = 1$ and $p = 2$. When $p = 1$, the p -stable distribution is the Cauchy distribution, when $p = 2$, the distribution is the Normal distribution.

Although the p -stable distribution is only found when $p = 1$ and $p = 2$, it is also possible to use LSH to undertake the c -ANN search by conducting a transformation on the dataset [45]. In this paper, we only focus on the Euclidean space where $p = 2$. In [6], only one LSH function has some

guarantee of the probability, but to boost accuracy and to limit the candidate set size, LSH [6] requires k hash functions to build an LSH table, and all L hash tables are required to keep the theoretical guarantee. For an object p and a query point q , if q falls into the same bucket on the i_{th} function with p , then we say p collides with q on function h_i . If p collides with query q on all the k hash functions in a hash table, then p is considered as a c -NN candidate. The algorithm will stop if (i) there is a candidate p , $\|p, q\| \leq cR$; or (ii) $3L$ candidates have been accessed.

Discussion LSH was designed for (R, c) -NN queries which is simply a decision version of the c -ANN problem. Particularly, for a given distance R , the algorithm will build an index considering the distance and then will return an object within distance cR to query point q with at least a constant probability. But to solve the c -ANN problem, radius R cannot be determined at the beginning. Since the index of LSH is built on radius R , when the radius has to be changed to different values in the c -ANN search, LSH with a fixed bucket partitioning requires multiple indexes for different radii (e.g., $1, c, c^2, \dots$) because the radius needs to be expanded until enough candidates can be found. However, multiple indexes will lead to indexes of enormous sizes, which is the main drawback of LSH. It is almost impossible to solve a c -ANN search using LSH with a fixed bucket partitioning.

4 Our approach

In this section, we present our early-termination condition with the incremental LSH technique, which is I/O efficient with a rigorous theoretical guarantee. In Sect. 4.1, we briefly introduce the motivation of our approach. Section 4.2 describes our LSH algorithm for the c -ANN problem and Sect. 4.4 shows that our approach can be immediately extended to support the top k c -approximate nearest neighbor search (c - k -ANN).

First, we formally introduce three termination conditions which are used in the following sections.

- Normal termination condition (denoted by NT): In the LSH-based c -ANN algorithm, there is usually an upper bound βn which is the maximum calculation time of the distance between a data point to the query point. If the algorithm computes the distance for βn points, we say the algorithm is terminated by the normal termination condition. The normal termination condition is applied on all traditional LSH algorithms.
- Traditional early termination is denoted by TET. TET indicates that if there is a point o that satisfies $\|o, q\| \leq cr/w$ where r denotes the current bucket width and w denotes the initial bucket width, the algorithm will stop.

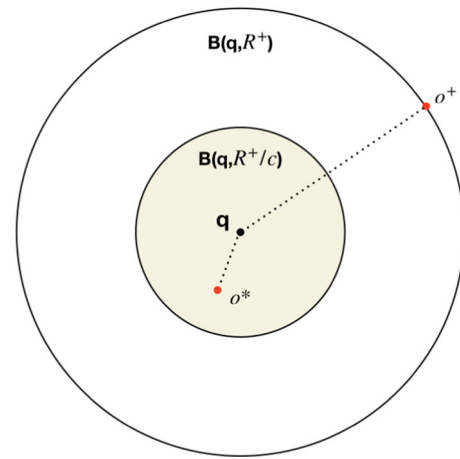


Fig. 2 Example: when ET could fail

- Aggressive early termination (denoted by AET) is the new early termination proposed in this paper. We precompute a parameter λ and if there is a point o which satisfies $\|o, q\| \leq \lambda r$ where r is the current projection distance between o and q , the algorithm will stop. The details will be explained in Sect. 5.

4.1 Motivation

For an object o and a hash function h_i , we use $d_i(o)$ to denote $|h_i(o) - h_i(q)|$, namely *projected distance* w.r.t h_i , which is the distance between o and q in the i -th projected dimension. Similar to other LSH schemes, a point o will be found as a candidate if o has been visited on enough hash functions.

In a list-based LSH algorithm, the early-termination condition is used when a “good enough” candidate has been found. For the TET condition, it only uses a very loose condition $R_{min} \leq cR$ to evaluate if the current best candidate is close enough, where R_{min} is the current smallest distance and R is the current radius. Once a point o with distance R_{min} satisfies the condition, it will be a correct c -ANN of q without any false probability according to the definition of c -ANN. However, if we hope to terminate the algorithm as early as possible without breaking the theoretical guarantee, calculating the probability that the current best candidate is a correct c -ANN could be a better idea. As shown in Fig. 2, suppose there is only one hash function. q is the query point and o^+ denotes the current nearest candidate to the query point q with distance R^+ . If o^+ is returned as the NN of q , the result is wrong if and only if there exists a point o^* which falls in the ball $B(q, \frac{R^+}{c})$ and $\|h(o^*), h(q)\| \geq \|h(o^+), h(q)\|$, in other words, o^* is not found as a candidate before o^+ .

In our AET condition, an important feature is the projection distances of each data point on all the hash functions. So we also propose an algorithm called incremental LSH to bet-

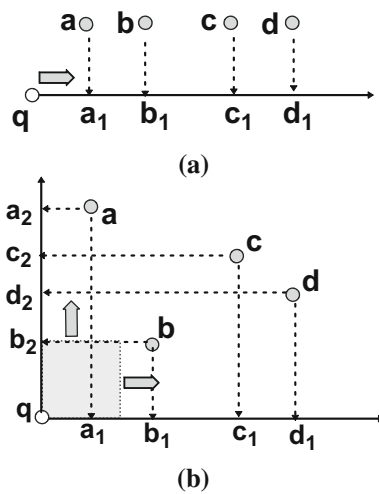


Fig. 3 Example for incremental search

ter support our AET. The key idea of our *Early termination driven Incremental LSH (EI-LSH)* method is to *incrementally* access the objects (i.e. IDs and their hash values) according to their projected distances, equipped with an AET condition. As shown in Fig. 3a, suppose there is one hash function h_1 and by a_1 , we denote the projected distance $d_1(a)$. In this example, a_1, b_1, c_1 and d_1 will be sequentially accessed. When there is more than one hash function (e.g., two hash functions h_1 and h_2 in Fig. 3b), the search will be conducted simultaneously with the same search radius. In Fig. 3b, the search region is a square. a_1 (i.e., ID of the object a and its hash value on h_1) will be accessed first, followed by b_2, b_1, d_2 , etc.

Given the incremental search strategy and the AET framework, the key challenge is to decide when the search should be terminated and how to identify a set of candidate objects such that the c -ANN can be returned with a theoretical guarantee. In Sect. 4.2, we propose our incremental LSH algorithm. Suppose there are m hash functions and n objects, an object becomes a candidate object if it has been accessed at least αm times, and we will have at most βn candidates, where α and β are pre-defined parameters according to the desired theoretical guarantee. The limitation of the candidate set could offer a guarantee on the I/O performance. On the other hand, we also devise the early termination technique based on the closest candidate object seen so far. We set a value of p_{ET} which demonstrates the probability that a real c -ANN has already be found at the current projection radius r , and then use this value to calculate a bound $R_{plus} = \lambda r$. We maintain the current best candidate o_{min} with a real distance R_{min} to the query point q , and compare R_{min} with R_{plus} . If R_{min} is smaller than R_{plus} , a value only related to p_{ET} and r , the AET condition will play a role and o_{min} will be returned as the result. As explained in Sect. 5, once p_{ET} is given, R_{plus} is only related to r and can be represented as

λr , using R_{plus} instead of calculating the probability for each radius r and a comparison with p_{ET} could avoid an integral operation which can be very expensive when it is repeated multiple times. The theoretical underpinning of our approach is provided in Sect. 5.

4.2 Incremental LSH with aggressive early termination

In this subsection, we describe our incremental LSH (EI-LSH) with the AET condition in detail. The setting of the parameters such as m, α, β and λ will be explained in the following theoretical analysis in Sect. 5. Our technique consists of two parts: *indexing* and *query processing*.

4.2.1 Indexing

The indexing step is similar to most LSH algorithms. In the indexing part, the high-dimensional dataset is projected to a low-dimensional space using a set of LSH functions. More specifically, for a given d -dimensional data D with n objects, we use m 2-stable hash functions to randomly project each object o into m hash values, denoted by $h_i(o)$ for the i -th hash function. Particularly, we have $h_i(o) = \mathbf{a}_i \cdot \mathbf{o}$ where $\mathbf{a}_i$ is randomly chosen from the normal distribution $\mathcal{N}(0, 1)$ (i.e., 2-stable distribution). For the i -th random projection, we use a B+ tree to keep the pair $(ID(o), h_i(o))$ for each object, where $ID(o)$ is the ID of the object o and the hash value $h_i(o)$ is the search key. After indexing, the d -dimensional dataset is mapped to an m -dimensional dataset where $m \ll d$. A good property of LSH is that the required hash function number m is not affected by dimensionality d , so even if the dataset is in a really high-dimensional space, m can still be a small value. Because of the good performance of B+ tree, our index can easily handle large-scale data and support the insert/deletion of the objects in an I/O efficient way. As shown in Sect. 5, we need to choose a proper m for the desired approximate ratio c and success probability δ . Let m_{max} denote the maximal m value we will support, we build the index with m_{max} B+ trees, denoted by $\mathcal{B}$, and randomly choose m B+ trees from the forest to do the search. Note that we also use a B+ tree to maintain the objects in $\mathcal{R}^d$ space, where the object ID is the search key.

4.2.2 Query processing

In general, a query object q in d -dimensional space will also be mapped into m projected dimensions; then the objects and their hash values will be incrementally accessed according to their projected distances in m projected dimensions. According to the LSH property, if a point o is close to q on most LSH functions, it is likely to be a c -ANN of q . So an object becomes a candidate if αm hash values of the point have been

Algorithm 1: EI-LSH ($\mathcal{B}, q$)

Input : $\mathcal{B}$: m B+ tree indexes for object IDs and hash values;
 q : the query object;
Output : o : the c -ANN object

```

1  $n_{can} := 0; d_{min} := 0;$ 
2 Apply  $m$  hash functions on  $q$ ;
3 while  $n_{can} < \beta n$  do
4    $o \leftarrow$  next object with smallest projected distance;
5    $i \leftarrow$  the projection dimension  $o$  comes from;
6    $r \leftarrow |h_i(o) - h_i(q)|$ ;
7    $cn(o) := cn(o) + 1$ ;
8   if  $cn(o) == \alpha m$  then
9     compute  $\|o, q\|$  and update  $d_{min}$ ;
10     $n_{can} := n_{can} + 1$ ;
11 if  $R_{min} \leq \lambda r$  then
12   break;
13 return  $o_{min}$ 

```

accessed. The search will terminate if there are already βn candidate objects or the AET condition is satisfied. Note that α , β and λ are pre-defined parameters, and their settings are discussed in Sect. 5.

Algorithm 1 presents the pseudo-code of our incremental LSH technique. We use n_{can} to record the number of candidate objects seen so far and o_{min} is the candidate object which is closest to q (Line 1). The query q will be mapped to m hash values (Line 2). Lines 3–13 conduct an incremental search according the projected distances of the objects. Note that an object may appear m times. During the search, Line 4 continues to retrieve the next closest object o with smallest projected distance, say from the i -th hash function, (Line 5) and the search radius r is set to the projected distance (i.e., $|h_i(o) - h_i(q)|$ at Line 6). Regarding the example in Fig. 3b, we have $o \leftarrow a$ and $i \leftarrow 1$ in the first iteration, and $o \leftarrow b$ and $i \leftarrow 2$ in the second iteration. Then, Line 7 increases the number of visited times of o , denoted by $cn(o)$, by one. The object o becomes a candidate object if $cn(o)$ reaches αm . Then, we compute its distance to q in $\mathcal{R}^d$ space (i.e., $\|o, q\|$) and d_{min} may be updated by o , where d_{min} keeps the closest candidate object seen so far (Lines 9–10). The incremental search will terminate in two cases: (1) we already have βn candidate objects (Line 3); or (2) the AET condition is satisfied based on the current d_{min} and search radius r (Lines 12–13). Finally, o_{min} is returned as the c -ANN, which is the closest candidate object.

Compared with the previous B+ tree-based LSH work QALSH, there are two key differences: (1) when the traditional termination conditions are not satisfied, QALSH chooses the next bucket on a B+ tree randomly and then visits all of the points in this bucket, while EI-LSH only chooses one point which is the closest to the query point over all B+ trees. So in every iteration, QALSH has to visit thousands of points and then check whether the termination conditions

are satisfied, but EI-LSH only needs to visit one point so that it avoids wasting time and I/O; (2) the early-termination conditions are different in QALSH and EI-LSH. With more aggressive early termination condition, EI-LSH can terminate the search procedure earlier every time under the given theoretical guarantee, which reduces the time and I/O cost.

Correctness For the given query q , approximate ratio c , and success probability δ , Sect. 5 shows that we can choose proper m , α , λ , and β values such that our EI-LSH algorithm can return c -ANN with a probability of at least δ .

CPU costs The dominant CPU costs in Algorithm 1 are the computation of distance in $\mathcal{R}^d$ space (Line 10) and the retrieval of the next object in the incremental search against m projected dimensions (Line 4). As the hash values are readily sorted by a B+ tree for each projected dimension, the bi-direction search can be conducted after the position of $h_i(q)$ is identified. A min-heap can be used to maintain $2m$ closest objects in each search direction, taking $O(\ln(2m))$ time to conduct an incremental search. Thus, the CPU costs of EI-LSH search are $O(s(d + \ln(2m)))$ where s is the number of iterations at Lines 3–13.

I/O costs The dominant I/O costs also come from the computation of distance (Line 10) and the incremental search (Line 4). Both hash values and object data IDs are organized by B+ trees, and we use the number of leaf nodes visited to evaluate the I/O costs because it is the dominant cost. We use I_{seq} and I_{ran} to denote the unit cost of sequential I/O read and random I/O read, where I_{seq} is much cheaper than I_{ran} in practice. To compute the distance between o and q , one random I/O will be invoked to load the object with d coordinate values. For the incremental search, as the hash values are sorted in B+ trees, we can retrieve the hash values and the corresponding object IDs in a pre-fetch fashion; that is, we use one random I/O to identify the position and apply l sequential I/O to load l pages. Thus, the total I/O costs are $O\left(\frac{s}{n_e} \cdot I_{seq} + \frac{s}{n_e \cdot l} \cdot I_{ran} + n_{can} \cdot I_{ran}\right)$ where n_e is the average number of entries per page,¹ n_{can} is the number of candidate objects accessed, and s is the number of iterations (i.e., the number of hash values visited).

4.3 Incremental LSH with traditional ET

Our algorithm can also adopt the traditional early termination condition which is safer. There are also two steps in I-LSH: indexing and querying. The indexing step is the same as EI-LSH, while the querying step is slightly different. Instead of comparing R_{min} , the current shortest distance to q , with λr , I-LSH compares R_{min} with cR where $R = \frac{r}{w/2}$. Since R is the current radius of $B(q, R)$, if an object o_{min} satisfies

¹ Note that each entry takes 8 bytes for one hash value and the object ID.

$R_{min} < cR$, it is safe to terminate the querying and return o_{min} as the result.

Algorithm 2: I-LSH($\mathcal{B}, q$)

Input : $\mathcal{B}$: m B+ tree indexes for object IDs and hash values;
 q : the query object;
Output : o : the c -ANN object

```

1  $n_{can} := 0; d_{min} := 0;$ 
2 Apply  $m$  hash functions on  $q$ ;
3 while  $n_{can} < \beta n$  do
4    $o \leftarrow$  next object with smallest projected distance;
5    $i \leftarrow$  the projection dimension  $o$  comes from;
6    $r \leftarrow |h_i(o) - h_i(q)|;$ 
7    $cn(o) := cn(o) + 1;$ 
8    $R \leftarrow \frac{2r}{w};$ 
9   if  $cn(o) == \alpha m$  then
10    compute  $\|o, q\|$  and update  $o_{min}$ ;
11     $n_{can} := n_{can} + 1;$ 
12   if  $R_{min} \leq cR$  then
13     break;
14 return  $o_{min}$ 

```

4.4 Extension of the c - k -ANN problem

In many real-life applications, in addition to the nearest neighbor, users are interested in the k nearest neighbors. In this paper, we also study the problem of a c -approximate k nearest neighbor (c - k -ANN) search. We say an object o is a correct result of a c - k -ANN search if $\|o, q\| \leq c \|o_k, q\|$, where o_k is the k -th nearest neighbor of q in objects $\mathcal{D}$.

Algorithm 1 can be easily extended to solve the c - k -ANN problem by applying the following changes: (1) instead of βn , we need to access $\beta n + k - 1$ candidate objects; (2) instead of o_{min} , we maintain the k -th closest candidate object o_k , and its distance to q will be used for early termination test; and (3) the k most-closest candidate objects will be returned as the result of the c - k -ANN search.

5 Theoretical analysis

In this section, we prove that if the AET framework is used on a proper LSH method, the algorithm can still have a theoretical guarantee. Furthermore, we show the correctness of our incremental LSH method; that is, for any given query q , approximate ratio c ($c > 1$) and success probability δ ($0 \leq \delta \leq 1$), our method can return a c -ANN with a probability of at least δ .

Before the formal proof, we stress some important frequently used notations. By o^* , we denote the NN object with distance R^* . By r , we denote the current search radius in the projected dimensions, which corresponds to the anchored

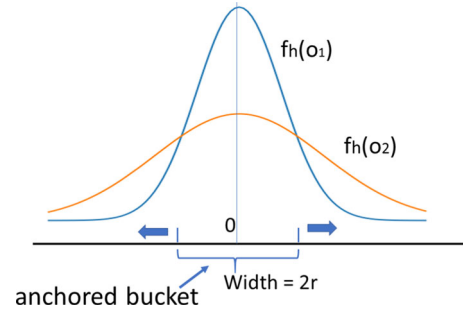


Fig. 4 Example of $f_h(x)$

bucket with width $2r$ and a ball $B(q, R)$ in high-dimensional space $\mathcal{R}^d$, where $R = \frac{2r}{w}$ and w is the initial bucket width. The object o_{min} , which is the closest candidate object seen so far, will be returned as c -ANN in Algorithm 1. We use R_{min} to denote its distance to q , and we say o_{min} is correct if $R_{min} \leq cR^*$.

Since the correctness of the AET framework is also related to the correctness of EI-LSH, we first prove without AET that EI-LSH can return a correct c -ANN with a success probability of δ .

5.1 Incremental increase of search radius r

Similar to other LSH-based methods, the correctness of our proposed method also relies on the (r_1, r_2, p_1, p_2) -sensitive property of the hash function. The key difference is that we incrementally increase the search radius r in Algorithm 1, following the distance of the hash values w.r.t the projection of q , i.e., $|h_i(o) - h_i(q)|$ with $1 \leq i \leq m$.

In this paper, we use the 2-stable distribution to construct the hash function h . Let $f_h(o)$ denote the distance between the object o and the query q in the projected dimension resulting from the hash function h , where $f_h(o) = |h(o) - h(q)|$. As shown in [6], the following lemma indicates that the $f_h(o)$ follows the normal distribution related to the distance between o and q in $\mathcal{R}^d$ space.

Lemma 1 For an object o with $\|o, q\| = \zeta$, $f_h(o)$ follows the normal distribution as $\zeta \mathbf{Z}$ where $\mathbf{Z}$ is a random variable drawn from $\mathcal{N}(0, 1)$ with density function $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$.

Suppose we have $\|o_1, q\| = 1$ and $\|o_2, q\| = 2$ Fig. 4 illustrates the distribution of $f_h(o_1)$ and $f_h(o_2)$, respectively. Given the bucket width w and centered at $h(q)$, we use $\eta(R, w)$ to denote the probability that o and q fall in the same bucket with $\|o, q\| = R$, which is the probability mass $\int_{-\frac{w}{2R}}^{\frac{w}{2R}} \phi(x) dx$.

In this paper, our LSH function is query-aware because we guarantee that the bucket, namely *anchored bucket*, is always centered at $h(q)$, in the projected dimension. We say the hash function h is (r_1, r_2, p_1, p_2) -sensitive with regard

to bucket width w if we have $Pr(|h(o) - h(q)| \leq \frac{w}{2}) \geq p_1$ given $\|o, q\| < r_1$ and $Pr(|h(o) - h(q)| \leq \frac{w}{2}) \leq p_2$ given $\|o, q\| > r_2$. The following lemma shows that a $(1, c, p_1, p_2)$ -sensitive hash function with bucket width w can become $(\xi, c\xi, p_1, p_2)$ -sensitive if the bucket width is set to ξw .

Lemma 2 *Given a hash function h , if it is $(1, c, p_1, p_2)$ -sensitive w.r.t bucket width w , then it is $(\xi, c\xi, p_1, p_2)$ -sensitive if the bucket width is set to ξw for any real value $\xi > 0$.*

Proof According to the definition of (r_1, r_2, p_1, p_2) -sensitive hash function, for bucket width w , we have

$$p_1 = \eta(1, w) = \int_{-\frac{w}{2}}^{\frac{w}{2}} \phi(x) dx$$

$$p_2 = \eta(c, w) = \int_{-\frac{w}{2c}}^{\frac{w}{2c}} \phi(x) dx$$

Then for bucket width ξw , we have

$$\eta(\xi, \xi w) = \int_{-\frac{\xi w}{2\xi}}^{\frac{\xi w}{2\xi}} \phi(x) dx = \int_{-\frac{w}{2}}^{\frac{w}{2}} \phi(x) dx = p_1$$

$$\eta(c\xi, \xi w) = \int_{-\frac{\xi w}{2c\xi}}^{\frac{\xi w}{2c\xi}} \phi(x) dx = \int_{-\frac{w}{2c}}^{\frac{w}{2c}} \phi(x) dx = p_2$$

Therefore, function h is $(\xi, c\xi, p_1, p_2)$ -sensitive with bucket width ξw for any $\xi > 0$. $\square$

According to Lemma 2, it is immediate that any given bucket width ξw corresponds to a ball $B(q, R)$ in $\mathcal{R}^d$ centered by q with $R = \frac{\xi}{2}$ space such that for any object $o \in B(q, R)$, $h(o)$ will fall in the anchored bucket (i.e., $|h(o) - h(q)| \leq \frac{\xi}{2}$) with a probability of at least p_1 , while for any object $o \notin B(q, cR)$, it will collide with q with probability less than p_2 .

Let r denote the current search radius in the projected dimensions (i.e., the width of the corresponding anchored bucket is $2r$). As ξ can be any non-negative real value in Lemma 2, the search radius r can be any real value with $r > 0$, and the hash function is $(\xi, c\xi, p_1, p_2)$ -sensitive where $\xi = \frac{2r}{w}$.

5.2 Correctness of the algorithm

Algorithm 1 may be terminated by a normal termination condition (denoted by NT) or an aggressive early termination condition (denoted by AET), and both may be correct

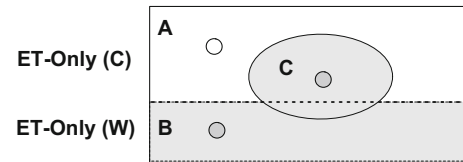


Fig. 5 The sample space $\mathcal{S}$

with some degree of probability. Thus, we need to carefully consider the correlation of the two termination conditions. Suppose the LSH algorithm is stopped by termination condition 1 (NT) with γ probability, and is terminated by AET condition with $1 - \gamma$ probability, the final success probability is $\gamma \cdot P_{NT} + (1 - \gamma) \cdot P_{ET}$, where P_{NT} and P_{ET} are the success probabilities of NT and AET separately. Our goal is to choose a set of parameters so that the final success probability δ is a constant value. First, we assume that the NT condition and the AET condition share a false probability and prove the correctness of the algorithm. Furthermore, we discuss what is the real proportion of the false probability for the two termination conditions. Below, we present the key idea of our proof, followed by the details illustrated in Fig. 5. (1) *Outline of the proof* Let $\mathcal{S}$ denote the sample space resulting from the m hashing functions, and each sample is an instance of the projections for the objects and the query. We first partition $\mathcal{S}$ into two disjoint two sets A and B . Suppose we only consider the aggressive early termination condition in Algorithm 1, denoted by ET-Only, the instance in A means that ET-Only will terminate with a correct o_{min} . Other instances belong to set B . Due to the use of the normal termination condition in Algorithm 1, some instances in A may be wrongly terminated by NT. Let C denote the set of instances in which an instance² is wrongly terminated by NT; that is, there are βn “bad” candidate objects and the AET condition is not satisfied. Note that we say a candidate is “bad” if its distance is larger than cR^* . Note that if both conditions are satisfied at the same time, the ET-Only algorithm is not interrupted by NT because o_{min} will be returned anyway.

Let $P(B)$ and $P(C)$ denote the probability mass of the samples in B and C , respectively. If we have $P(B) + P(C) \leq 1 - \delta$, then Algorithm 1 will return the correct c -ANN object with a probability of at least δ .

(2) *Parameter settings* The desired accuracy in this paper is defined by c and δ , where δ is the success probability and c is the approximate ratio. Similar to other LSH papers, we set $\delta = 1/2 - 1/e$. Note that δ actually can be boosted by repeating the procedure and taking the median of the results. For the initial bucket width w , we set w to 3.5. Then, we can calculate probability p_1 and p_2 following the Lemma 1 with $R = 1$ and $R = c$, respectively. According to Lemma 2, it is

² Here, it is not necessary that the instance belongs to A .

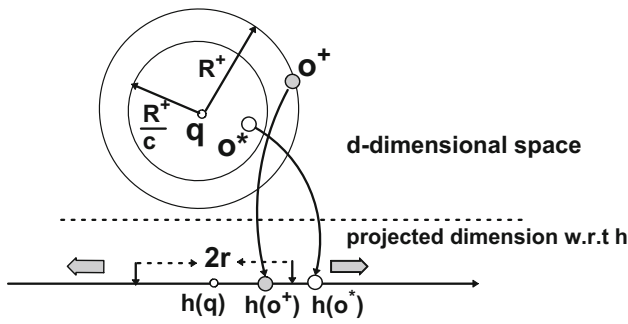


Fig. 6 Motivation of early termination

immediate that the hash function is (R, cR, p_1, p_2) -sensitive regarding the search radius r with $R = \frac{2r}{w}$.

In this paper, let $\delta = 1/2 - \delta'$ (so when $\delta = 1/2 - 1/e$, $\delta' = 1/e$), we set $\beta = 0.01$ and $\alpha = \frac{\sqrt{\frac{\ln \frac{2}{\beta}}{\ln \frac{1}{\delta'}} p_1 + p_2}}{1 + \sqrt{\frac{\ln \frac{2}{\beta}}{\ln \frac{1}{\delta'}}}}$ where $p_1 < \alpha < p_2$, and $m = \left\lceil \frac{\ln \frac{1}{\delta'}}{2(p_1 - p_2)^2} \left(1 + \sqrt{\frac{\ln \frac{2}{\beta}}{\ln \frac{1}{\delta'}}} \right)^2 \right\rceil$. For aggressive early termination, we set $P_E = \frac{1-\delta}{2}$ and the calculation of λ will be introduced in the following part.

(3) *Computation of $P(B)$ for aggressive early termination only* Now we show that, if we only use the AET condition in Algorithm 1, we have $P(B) \leq P_E$. Based on Lemma 1, it is immediate that the probability of hitting the anchored bucket will monotonically decrease when the distance between the objects and the query grows, which is formally described as follows.

Lemma 3 Given two objects o_1 and o_2 where o_1 is closer to q than o_2 , with distance R_1 and R_2 , respectively, we have $\eta(R_1, w) \geq \eta(R_2, w)$. Recall that $\eta(R, w)$ denote the probability that an object o with distance R falls in the anchored bucket with width w .

As shown in Fig. 6, o^+ denotes the closest candidate object seen so far with distance R^+ . If we simply return o^+ as the c -ANN object, the result is *incorrect* if $R^+ \leq \frac{R^+}{c}$ (i.e., o^+ falls in the ball $B(q, \frac{R^+}{c})$ and o^* is not a candidate object (e.g., $|h(o^*) - h(q)| > r$ for $m = 1$). Thus, for the given r, m, c , and probability P_E , we arrive at a distance R^+ such that, with probability $P_{ET} = 1 - P_E$, an object o with distance $\frac{R^+}{c}$ will become a candidate w.r.t search radius r . Specifically, for the given P_E and m , we find a probability p such that $P_E = 1 - \sum_{i=\alpha m}^m C_m^i p^i (1-p)^{(m-i)}$. That is, for an object with distance $\frac{R^+}{c}$, it will have probability p to hit an anchored bucket w.r.t r on a single hash function, and hence becomes a candidate (i.e., hit at least αm times) with probability $1 - P_E$ on m independent 2-stable hashing functions. With the given p and r , we can calculate a distance $R^+ = \lambda r$ such that

$\eta\left(\frac{\lambda r}{c}, 2r\right) = p$. Particularly, we have $\lambda = \frac{c}{N^{-1}\left(\frac{p+1}{2}\right)}$ and $N^{-1}(\cdot)$ is the inverse function of the cumulative distribution function of the standard normal distribution. Note that the calculation of λ is not related to search radius r , and hence λ can be pre-computed based on P_E, c , and m .

Then, we have the following lemma showing that $P(B) \leq P_E$.

Lemma 4 Algorithm 1 with early termination only will return an incorrect c -ANN with probability at most P_E ; that is, we have $P(B) \leq P_E$;

Proof According to Lemma 3 and the calculation of R^+ , if the early termination condition is satisfied (i.e., $R_{min} \leq R^+$), the probability that the returned o_{min} is not a c -ANN is at most P_E . That is, $P(B) \leq P_E$. $\square$

(4) *Computation of $P(C)$* In this part, we show that $P(C) \leq \frac{1-\delta}{2}$. Recall that for each instance in C , Algorithm 1 is terminated by the NT condition with an incorrect o_{min} meanwhile the aggressive early termination condition is not satisfied; that is, we have: (1) βn “bad” candidates; and (2) $R_{min} > R^+$.

Lemma 5 With probability at most $\frac{1-\delta}{2}$, Algorithm 1 is terminated by the normal termination condition with an incorrect c -ANN; that is, we have $P(C) \leq \frac{1-\delta}{2}$.

Proof Given that the AET condition is not satisfied at search radius r , which implies that none of the objects within ball $B(q, R^+)$ are chosen as candidates, now the algorithm is stopped by the NT condition, which means there have already been βn candidate objects outside of ball $B(q, R^+)$. According to the setting of P_E, m and the calculation of R^+ , we have $R^+ > cR$. Thus, all these candidate objects are also outside of ball $B(q, cR)$.

Given an object outside of ball $B(q, cR)$, we use p_3 to denote the probability that o is chosen as a candidate (i.e., $\Pr(cn(o) \geq \alpha m)$) where $cn(o)$ is number of hits of the anchored bucket with width $2r$. According to the Chernoff bound, we have $p_3 < \exp(-2(\alpha - p_2)^2 m)$ because our LSH function is (R, cR, p_1, p_2) -sensitive for any R according to Lemma 2. Let X denote the number of candidate objects outside of ball $B(q, cR)$ (i.e., number of false positives). According to the Markov Inequality, we have $P_N = \Pr(X \geq \beta n) \leq \frac{n \cdot p_3}{\beta n} = \frac{p_3}{\beta}$. Since $\frac{p_3}{\beta}$ can be calculated with given parameters, the probability of AET with βn candidates is bounded by $\frac{1-\delta}{2}$. Therefore, $P(C)$ is bounded by P_N because the probability of “visiting βn bad candidates and AET not satisfied” is bounded by that of “visiting βn candidates and AET not satisfied”. $\square$

(5) *Correctness of algorithm* Based on the above analysis, the correctness of Algorithm 1 follows since $P(B) + P(C) \leq 1 - \delta$.

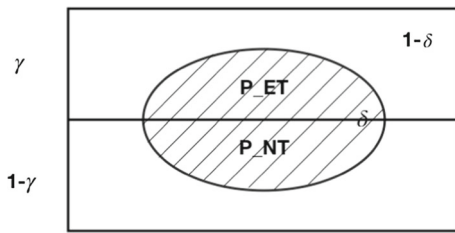


Fig. 7 Success probability δ

Theorem 1 When Algorithm 1 terminates with object o_{min} , we have $\|o_{min}, q\| \leq cR^*$ with probability at least δ , where R^* is the distance of NN object o^* to q in $\mathcal{R}^d$ space.

(6) *Discussion of P_B and P_C* We suppose P_B and P_C have the same value to simplify the proof, and in this section, we discuss whether it is possible to allow P_B to be a larger value so that the early-termination condition can be more efficient. In Fig. 7, we can obtain the final success probability $\delta = P_{ET} \cdot (1 - \gamma) + P_{NT} \cdot \gamma$ where P_{ET} , P_{NT} , γ demonstrate the success probability of early termination ($P_{ET} = 1 - P_B$), the success probability of NT ($P_{NT} = 1 - P_C$), and the probability that the algorithm is terminated by NT separately. If we can prove that γ is a value smaller than 50% (the value we set in the proof above, which means ET and NT divide the false probability equally), P_B could be set to a larger value and the total success probability will not be changed. Here, we discuss the bound of γ .

With the given p which affects the success probability of the aggressive early termination condition P_{ET} , we can compute the stop probability of the normal termination (NT) γ as following:

When we are performing the c -ANN search, if the search is terminated by AET, it will satisfy $R_{min} \leq \lambda r$ where $\lambda = \frac{c}{N^{-1} \binom{p+1}{2}}$. Therefore, if it is stopped by the normal termination condition, we have $R_{min} > \lambda r$ for all candidates which have been assessed for at least αm times. Let p_{can} denote the probability that a point o , whose distance to q is slightly greater than λr , has been found as a candidate, we have $p_{can} = \sum_{i=\alpha m}^m C_m^i p^i (1-p)^{m-i}$. The normal termination condition will work if and only if all of the βn candidates found do not satisfy the AET condition, which means all the candidates o have a distance to q larger than λr . It is easy to get γ , the probability that NT terminates the algorithm, is $p_{can}^{\beta n}$.

Because the success probability for the NT condition only is at least δ , which has been proved before, we have

$$\begin{aligned} (1 - \gamma)P_{ET} + \gamma \cdot \delta &\geq \delta \\ (1 - \gamma)P_{ET} &\geq \delta - \gamma \cdot \delta \\ (1 - \gamma)P_{ET} &\geq (1 - \gamma)\delta \end{aligned} \quad (1)$$

As previously mentioned, the relationship between p and P_{ET} is $P_{ET} = p_{can} = \sum_{i=\alpha m}^m C_m^i p^i (1-p)^{m-i}$, so from the above probabilities, because $0 < \gamma < 1$, we will get a total success probability greater than δ as long as we choose a proper p to have $P_{ET} > \delta$. The theoretical guarantee still holds.

Compared with the formal setting $P_E = \frac{1-\delta}{2}$, the new P'_E has a lower bound of $1 - P_{ET} \leq 1 - \delta$, which is a larger value of P_E . A greater P_E leads to a greater R^+ , so the algorithm can stop earlier without breaking the theoretical guarantee. In conclusion, EI-LSH with AET can find the required c -ANN results under the given theoretical error bound.

5.3 Comparison with C2LSH and QALSH

Although our technique follows the framework of C2LSH and QALSH, we show that the AET framework and the incremental search strategy underpinned by the above analysis can bring us the following benefits.

(1) *More promising early termination technique* C2LSH and QALSH use the traditional early termination condition where the algorithm can be terminated with $R_{min} \leq cR$. This decision is *safe* (i.e., deterministic) because it guarantees to return a correct c -ANN in their algorithms, and hence they do not need to consider its correlation with a NT condition. But the safe early termination (TET) condition also causes an extra success probability to the theoretical bound. Our strategy is much more aggressive as we have $R^+ > cR$, and hence can significantly reduce the number of candidate objects visited in practice, at a cost of a more complicated proof to handle the correlation between two termination conditions. In this section, we carefully prove that EI-LSH with a new AET condition can find the c -ANN with the success probability δ .

(2) *Better approximate ratio* As shown in [18], C2LSH and QALSH return a c^2 -approximate NN if the approximate ratio is set to c in their algorithms. This is because their bucket width grows to $w c^k$ during the search where k is an integer, and $R_{min} \leq cR$ and $R \leq cR^*$ when their search algorithms terminate, where R^* is the NN distance and R is the corresponding ball radius in $\mathcal{R}^d$ for search radius r . This implies that the value of c has to be set to $\sqrt{c}$ to achieve c -approximate in their algorithms. Thanks to our incremental search strategy and the novel early termination condition, Algorithm 1 and Algorithm 2 can directly use c as the parameter to achieve c -approximate NN with a success probability of at least δ . Thus, under the same theoretical guarantee, the number of hashing functions required by EI-LSH is much smaller than that of C2LSH and QALSH.

6 Evaluation

In this section, we conduct comprehensive experiments to demonstrate the I/O efficiency of our proposed algorithm, compared with two state-of-the-art I/O efficient c -ANN search algorithms.

6.1 Experiment setup

In this subsection, we present the experiment settings of our performance evaluation.

Benchmark methods SRS [37] and QALSH [18] are two state-of-the-art I/O efficient c -ANN search algorithms. In [18], QALSH has demonstrated superior I/O performance compared with C2LSH [11] and LSB-tree [38], hence we exclude the C2LSH and LSB-tree from the performance evaluation. Furthermore, we also compare EI-LSH and I-LSH to show the effect of our early termination framework.

- **QALSH** uses a set of single LSH functions for the c -ANN search. It requires m B-trees to store the index. The source code of QALSH is provided by the authors. Note that, in their implementation, the authors only keep the minimal hash value for each B+ tree leaf-node, together with all object IDs in the leaf-node. This can significantly reduce the index size.
- **SRS** has several variants. In the performance evaluation, we use the I/O efficient version with a theoretical guarantee, where objects are indexed by an m -dimensional R-tree based on their hash values. The source code of SRS is publicly available at <https://github.com/DBWangGroupUNSW/SRS>.
- **I-LSH** is our incremental search LSH algorithm proposed in this paper without the new early termination condition (Algorithm 2).
- **EI-LSH** is our incremental search LSH algorithm proposed in this paper (Algorithm 1 in Sect. 4) using the new early termination condition.

Similar to the previous work, we evaluate the performance on the c - k -ANN version of these algorithms in the experiments where k varies from 1 to 100, with a default value of 40.

Datasets We use four million-scale datasets in the experiments, which are downloaded from a recent nearest neighbor search benchmark.³ Since all of the three methods are available to deal with real numbers, we did not scale up the data.

- **Tiny** contains around 5 million GIST feature vectors with dimensionality 384.

- **Million Song** is a collection of audio features and meta-data for a million contemporary popular music tracks with 420 dimensions.
- **Glove** contains 1.1 million 100-dimensional word feature vectors extracted from Tweets.
- **Sift** consists of 1 million 128-dimensional SIFT vectors
- **Sift 0.2billion** consists of 0.2 billion 128-dimensional SIFT vectors
- **Audio** is a 192-dimensional dataset with 50,000 points
- **Tiny_80m** is the 114 GB full tiny dataset containing 80 million 384-dimensional vectors.

Evaluation metrics In the experiments, following important metrics are evaluated to show the time, space and I/O cost efficiency of our proposed EI-LSH.

- **Index size** The index size is one of the important metrics to evaluate the performance of c -ANN search methods. We compared the index sizes of QALSH, EI-LSH and SRS, and also compared the numbers of hash functions (m) used in these three models for the desired theoretical guarantee.
- **Index building time** Although the indexes are only built once for each dataset, if building index takes too much time compared with other algorithms, it can also be a shortcoming. In order to evaluate the efficiency of EI-LSH, we compared the index building time between all evaluated methods. To fairly compare the algorithms, all three algorithms read data from text files instead of binary files.
- **I/O efficiency** We evaluate the I/O efficiency of the c -ANN algorithms under the same theoretical guarantee. Following the convention, we count the number of I/Os during the computation for three algorithms. As mentioned in Sect. 4.2, it is important to consider the difference between random I/O and sequential I/O. In many existing works (e.g., [3,5,7,9–12,23,26,35,39,43]) where both sequential and random I/Os are involved, the ratio $l = I_{seq}/I_{ran}$ has been widely used for I/O costs evaluation, where l ranges from 0.1 (e.g., [5]) to 0.0001 (e.g., [7]). In order to choose appropriate ratio l , we compared the access speed between sequential I/O and random I/O on both solid state drive (SSD) and hard disk drive (HDD) by reading the same file sequentially and randomly. As a result, the l is around 0.1 and 0.01 on SSD and HDD, respectively. In our experiment, default ratio l is set to 0.1 which is in favor of the baseline methods such as SRS, because all I/O reads in SRS are random, while both random and sequential I/Os are involved in QALSH and EI-LSH. We assume the non-leaf nodes of B+ tree and R-tree are resident in the main memory. Regarding the the accesses of leaf nodes, sequential I/O and random

³ https://github.com/DBWangGroupUNSW/nns_benchmark/tree/master/data.

Table 2 Index size

Dataset (d)	Glove (100)	Sift (128)	Tiny (384)	Million Song (420)	Audio (192)	SIft0.2b(128)	Tiny 80 million (384)
m	84	83	93	83	60	132	125
QALSH (MB)	444	394	2150	391	32	48,446	38,146
m	16	16	16	16	16	16	16
EI-LSH (MB)	165	149	735	157	12	12,207	9765
m	6	6	6	6	6	6	6
SRS (MB)	46	43	254	46	2.4	7988	3482

I/O are involved by B+ tree and R-tree, respectively. The data loading process from original dataset is considered as random I/O for all three algorithms.

- *Accuracy* We also evaluate the accuracy of the search results by reporting the ratios of returned results compared with the ground truth answers. The reported ratio c' is the average of ratios R'_i/R_i , where R'_i and R_i are the distances between the query point and i^{th} closest object in the returned results and ground truth answers, respectively.
- *Query answering time* The query answering time is the key metric for evaluating the efficiency of proposed EI-LSH. We thoroughly evaluate the query answering time on multiple datasets with different settings of approximation ratio c .

Parameter setting To compare the methods fairly, the success probability δ is set to $\frac{1}{2} - \frac{1}{e}$ for all algorithms. The default approximate ratio c is set to 4. Note that QALSH is a c^2 approximate method and hence needs to set c to $\sqrt{4.0} = 2$ to achieve the 4-approximation, while both SRS and I-LSH are c -approximate methods. We use the default settings of QALSH and SRS unless otherwise specified. Specifically, QALSH sets $\beta = \frac{100}{n}$, so at most $100 + k - 1$ candidate objects will be evaluated. As suggested, its initial bucket size is set to 2.72 and the number of required hash functions m is calculated accordingly. In SRS, we use a constant value of m for a given c , following its settings in [37]. For I-LSH, the parameters are set according to the specification of the parameter setting in Sect. 5. For our proposed EI-LSH, since the I/O cost increases with the increase of the value of m , we choose the lowest m value, i.e., $m = 16$ under the theoretical guarantee. The page size B is set to 8192 bytes for all the datasets and algorithms.

All of the experiments were conducted on a PC with Intel(R) Xeon(R) CPU E3-1231 v3 with 3.04GHz, 8 cores and 16G main memory. All of the source codes were implemented in C++ and compiled with g++ -with -O3 flag. We evaluate three algorithms by studying the performance after averaging the 200 queries. To compare the performance fairly, we use the same random vectors to project the datasets.

6.2 Index Size evaluation

We list the index size for all the datasets we have used in the experiments under the default settings (e.g., $c = 4$ and $\delta = \frac{1}{2} - \frac{1}{e}$). Table 2 shows the index sizes for QALSH, EI-LSH and SRS and the number of hash function (m) used for the desired theoretical guarantee. It is reported that SRS uses the smallest index size. This is because a small m value is required and SRS only keeps one data entry for each object. EI-LSH uses a much lower number of hash functions compared with QALSH, mainly because QALSH is a c^2 -approximation algorithm. The index sizes of the three methods are independent of the dimensionality d , and grow linearly with the dataset size n .

6.3 Index building time evaluation

Another important evaluation metric is the efficiency to build indexes for the LSH methods. Although the indexes are only built once for each dataset, if building an index takes too much time compared with the other algorithms, it can also be a shortcoming. To compare the algorithms fairly, all three algorithms read data from text files instead of binary files. We list the index building time for all the datasets we have used in the experiments under the default settings. Table 3 shows the indexing time cost for QALSH, SRS and EI-LSH. For list-based LSH algorithms, the indexing time is mainly dependent on the number of hash functions. According to our proof, EI-LSH needs less hash functions than QALSH to hold the same theoretical guarantee and building procedures of the two algorithms are similar. Therefore, for all the datasets, EI-LSH only costs about 25% of the time to build the index compared with QALSH. For the tree-based LSH methods, there are some different reasons to consider. Although m is less than 10 in SRS, the algorithm has to do the insertions on an m -dimensional tree, which is a more comprehensive operation compared with an insertion on a B+ tree. SRS has the smallest index size, but the slowest index building speed.

Table 3 Indexing time

Dataset	Tiny	Audio	Glove	Million Song	Sift	Sift0.2b	Tiny 80 million
SRS(s)	1777	9.42	568	421	490	141,798	28,512
QALSH(s)	386	2.71	67	153	56	11,892	6162
EI-LSH(s)	89	1.1	13	30	11	2332	1421

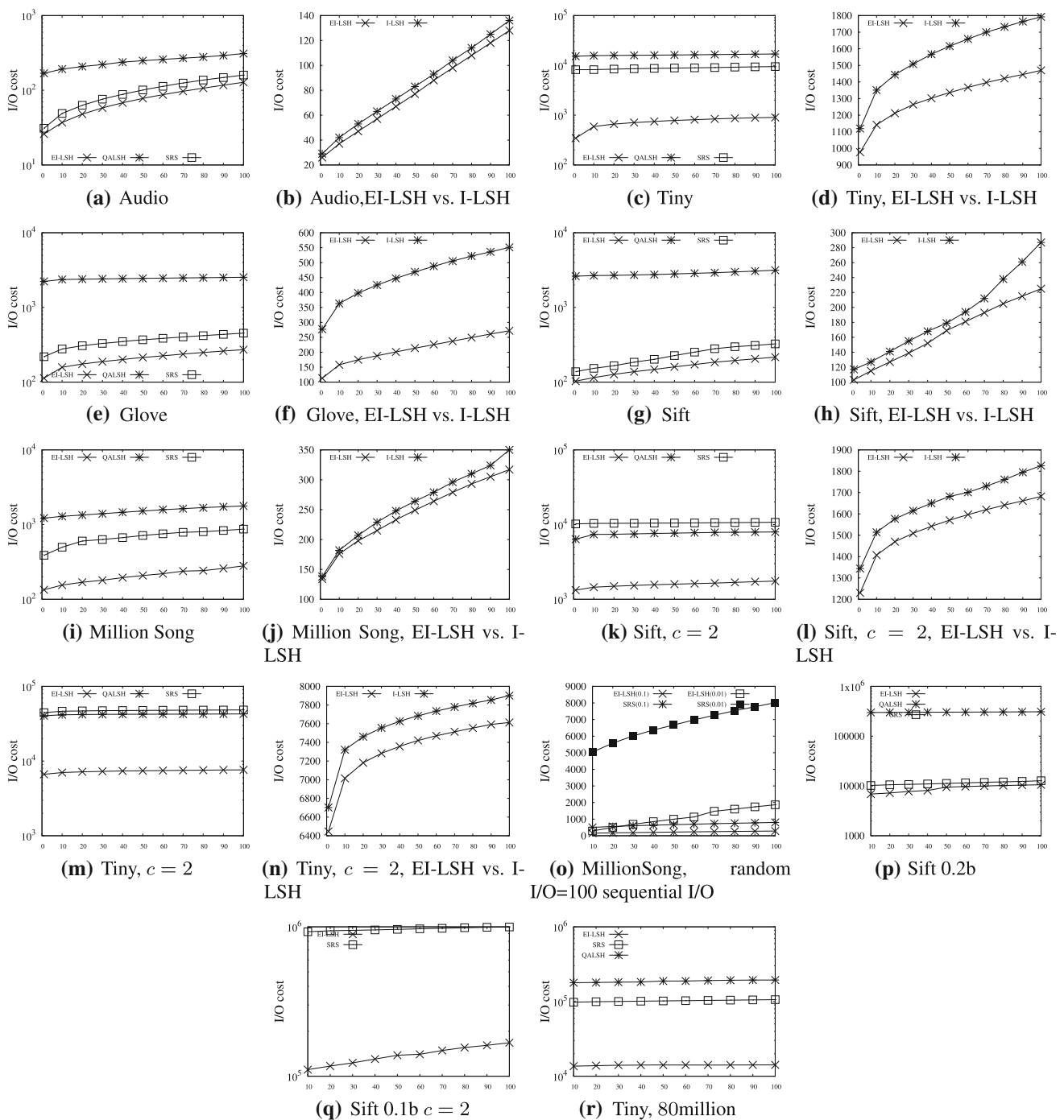
6.4 I/O costs evaluation

In this section, we evaluate the goodness of an I/O efficient c -ANN algorithm and the most essential criteria is to use the smallest I/O costs as possible under the same theoretical guarantee for the given approximate ratio c and success probability δ . The three algorithms undergo a rigorous theoretical analysis to ensure the desired theoretical guarantee, and all the queries in the experiments are answered *correctly* regarding the given approximate ratio c . Figure 8 reports the I/O costs of the three algorithms on four datasets where k varies from 1 to 100 and the approximate ratio c is set to 4 (default value) or 2, respectively. Furthermore, to prove that our new early termination framework can decrease the I/O cost dramatically, we also conduct experiments for EI-LSH and I-LSH only to demonstrate the results more clearly because both of the two methods use a much smaller I/O compared with SRS and QALSH. In our experiment, new early termination framework can help the model to stop the search process earlier every time, thereby reducing the I/O cost dramatically. From Fig. 8, we make the following observations:

- EI-LSH has the best I/O performance on the four datasets under all settings, especially on the million-scale and billion-scale datasets.
- When $c = 4$ (i.e., 4-ANN), QALSH is not competitive compared with SRS and EI-LSH mainly because it is set to $c = \sqrt{c}$ to achieve the same approximate ratio with SRS and EI-LSH, which causes a much larger m . SRS has the smallest index size, but because of the penalty of random I/O, SRS requires more I/O costs to find enough candidates. The performance gap between SRS and EI-LSH is not very significant on the SIFT dataset where the dimensionality is 128. On the Tiny and Million Song datasets with dimensionality of 384 and 420, respectively, EI-LSH outperforms SRS by a large margin.
- We also conducted experiment on the full Tiny dataset whose size is over 100 GB, and the SIFT dataset with 0.1 billion, 0.2 billion. On the Tiny dataset, the I/O cost of SRS is more than 8 times the I/O cost of EI-LSH. On the SIFT dataset, the performance gap between SRS and EI-LSH is similar to the gap between them on the smaller SIFT dataset with 1 million data points. But when c is set to 2, the I/O cost of SRS dramatically increases due

to the property of the R tree and EI-LSH outperforms SRS in this experiment. Since the I/O cost of QALSH on SIFT 0.1b ($c = 2$) is not comparable with the other two algorithms, we do not report it in the figure.

- When the approximate ratio c is set to 2, it is not surprising that the higher accuracy demand leads to much more expensive I/O costs for the three algorithms. As shown in Fig. 8, SRS is more sensitive to a decrease of c (i.e., an increase in the accuracy required) compared with QALSH and EI-LSH. Although SRS consistently outperforms QALSH when $c = 4$, it consumes more I/O costs when $c = 2$. This is because the performance of the exact NN search on R-tree is very sensitive to the dimensionality, and m of SRS is set to 15 for $c = 2$. Although the number of B+ trees also increases to a large number, e.g., $m = 60$ and $m = 351$ for EI-LSH and QALSH, respectively, on the Tiny dataset, their performance is less sensitive to an increase in the approximate ratio compared to SRS. On the large dataset, SIFT 0.2b, the I/O cost of QALSH increases to more than 200,000 due to its index size, while EI-LSH only costs about 7000 I/O since it requires less B+ trees.
- When k increases, the I/O costs also grow steadily for the three algorithms. It is noted that k does not significantly affect the I/O costs when the dataset size is large, because the dominant I/O costs of the three algorithms on a large dataset are to find the first candidate.
- The ratio of sequential I/O and random I/O does not have much effect on EI-LSH performance, and if we set the ratio to 0.01, the I/O cost of EI-LSH only increases from 300 to 1800 when $k = 100$ on MillionSong, but the I/O cost of SRS dramatically grows from 800 to 8000. The reason for this is that all of the I/O costs in SRS are random I/O so a small ratio of sequential I/O and random I/O will affect SRS more than EI-LSH.
- When only considering I-LSH and EI-LSH, EI-LSH can reduce the I/O cost on all the datasets under the same theoretical guarantee because it has a stricter bound of the success probability. If k grows to a large value, the I/O cost difference between I-LSH and EI-LSH becomes more obvious.

Fig. 8 I/O costs varying k

6.5 Accuracy evaluation

Our experiments show that all three algorithms are quite pessimistic. Given a very low success probability $\delta = \frac{1}{2} - \frac{1}{e} = 0.132$, all the queries of the three algorithms are answered correctly with a ratio smaller than 2.0 for the required ratio $c = 4.0$. Given the fact that all three algorithms rely on the same hash function, and the key differences are the search

strategy and the indexing method, suppose the same number of hash functions are deployed for three algorithms. Intuitively, the more candidate objects accessed by an algorithm, the better ratio it should be able to achieve given a reasonably good search heuristic. The key challenge is how to stop as early as possible under the theoretical guarantee. As expected, given the same c and δ , the ratio of QALSH is

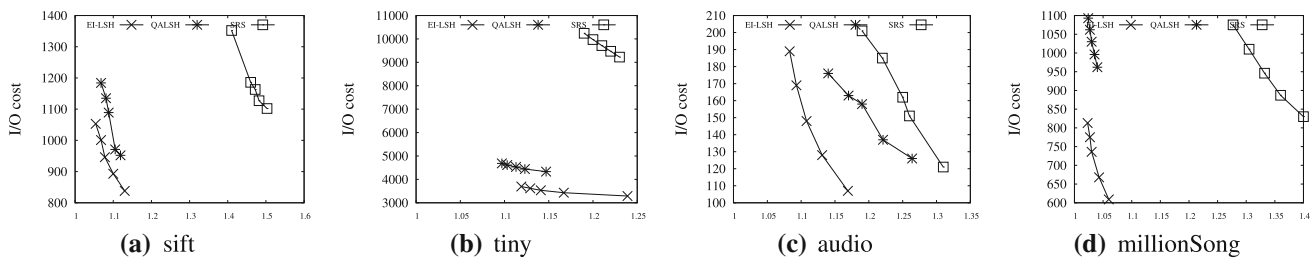


Fig. 9 I/O costs versus ratio

Fig. 10 I/O costs versus c

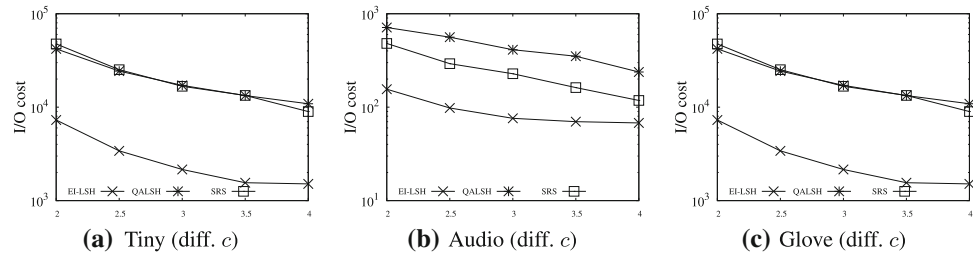
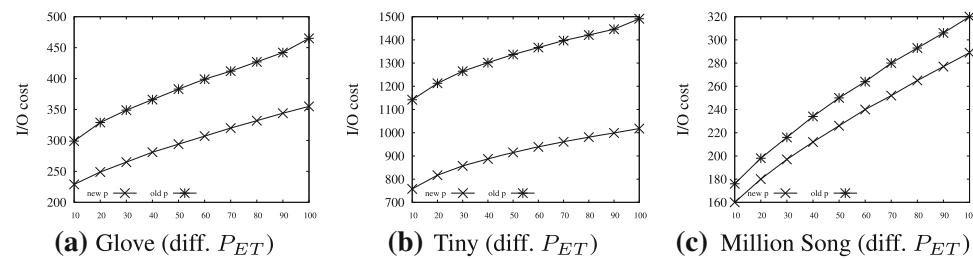


Fig. 11 I/O costs versus P_{ET}



much better than that of SRS and EI-LSH because QALSH terminates much later.

Instead of merely reporting the ratio versus k , in Fig. 9, we report the trade-off between I/O costs and the average approximate ratio of the queries for the three algorithms on the SIFT, Tiny, Audio and MillionSong datasets by forcing the algorithm to explore some objects (i.e., more I/O costs incurred) even when the early termination condition has been satisfied. It is shown that, under the same I/O costs, EI-LSH can achieve a lower ratio. Similarly, under the same ratio, EI-LSH incurs much lower I/O costs. The performance of SRS is not competitive in this set of experiments mainly because of the penalty of random I/O accesses and the R+ tree property. Another observation from this set of experiments is that, it is more difficult for SRS to return a high quality result even if it is forced to visit more points. From the figures, the ratio decreases slowly when the I/O cost increases. If accuracy is a requirement of the result, SRS is not competitive.

6.6 Effect of the approximate ratio c

A good c -ANN algorithm should support different approximate ratios, especially when c is a small value. We conduct experiments on Tiny and Million Song datasets where c varies from 2.0 to 4.0, and we evaluate the I/O costs. The

results are reported in Fig. 10. Similar to Sect. 6.4, the difference between EI-LSH and I-LSH are shown separately. of I/O than QALSH, but the I/O cost of SRS rises very fast, when $c = 2$, it becomes 140% of QALSH and about 7 times of EI-LSH. As expected, the performance of the three algorithms degrades with a decrease in the approximate ratio. It is shown that the performance of SRS is most sensitive to a change of the approximate ratio c . This is because a higher requirement of accuracy leads to a larger m , and hence deteriorates the performance of the exact NN search on R-tree in SRS.

6.7 Effect of the value of P_{ET}

As discussed in Sect. 5, as long as the value of P_{ET} is greater than δ , the theoretical guarantee still holds. We conduct experiments to illustrate the effect of adopting a stricter theoretical bound to the I/O cost. In the default setting, P_{ET} is set to $1 - \frac{1-\delta}{2} = 1/2 + \delta/2$ in EI-LSH and in this experiment, $P_{ET} - \text{new}$ is set to $\delta + 0.001$. When $c = 4$, λ should be 8.7 and 22.3, separately. The results are reported in Fig. 11. The 'old p' line demonstrates the former setting of P_{ET} and the 'new p' line denotes the new setting of P_{ET} . The effect of P_{ET} is greater on a higher dimensional dataset or on a larger dataset. This is because on a larger dataset or a higher

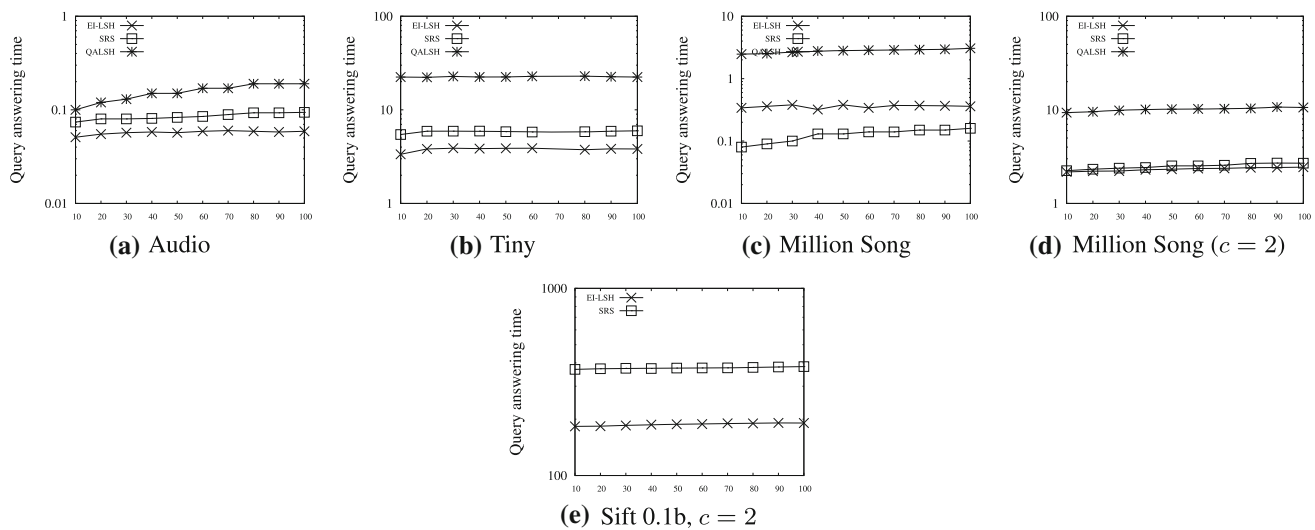


Fig. 12 Query answering time versus k

dimensional dataset, there is less probability of finding k - c -ANN within k candidates only and therefore, a small P_{ET} can terminate the algorithm earlier and reduce the I/O cost. On the million-scale datasets, the Tiny dataset is the largest one and shows the most obvious difference between the two P_{ET} . For any k in the figure, the I/O cost of the new p is at least 400 less than the old p .

6.8 Query answering time

In this section, we evaluate the query answering time of the three algorithms. Figure 12 shows that SRS and EI-LSH run much faster than QALSH since QALSH requires a larger index to hold the same theoretical guarantee. On Audio and Tiny, SRS costs about 2 times the running time compared with EI-LSH, while QALSH is the slowest. On the Million-Song dataset, when $c = 4$, SRS is faster than EI-LSH but when c is set to 2, the time consumption of SRS is higher than EI-LSH which proves that R+ tree does not support a large m very well. The limitation of R+ tree will be more obvious on a large dataset. When $c = 2$ and the dataset size is 0.2 billion, SRS costs more than 100 seconds per query on the SIFT dataset. For the experiment on SIFT 0.1 billion ($c = 2$), since QALSH runs much longer than the other two algorithms, we do not report it in the figure.

6.9 Scalability

Another important feature of a c -ANN algorithm is its scalability. We run our algorithm on the large dataset Tiny with different scales to evaluate the performance on a non-trivial dataset. Since QALSH's indexing time cost is unaffordable, we only compare SRS and our algorithm.

Table 4 Index size

Size of dataset (million)	10	30	50	80
m	16	16	16	16
EI-LSH (GB)	1.3	3.8	6.4	10.1
m	6	6	6	6
SRS (GB)	0.34	1.1	1.9	3

The following Table 4 shows the index sizes of EI-LSH and SRS for different size of the Tiny dataset.

We change the dataset size from 10 million to 80 million to show the I/O performance of our algorithm. The value of k is set to 50, and all the other settings are the same as other experiments. To better show the I/O efficiency of our ET framework and our incremental search strategy, we use the most strict bound of the success probability where $P_{ET} = \delta + 0.001$. Under this setting, in most cases, the first k candidates found by EI-LSH can satisfy the early-termination condition. Figure 13 shows the I/O cost for different sizes of the Tiny dataset. When the size grows to 80 million, the I/O cost of EI-LSH to solve the c - k -ANN problem is less than 6000, which is a small value considering the huge dataset size. Compared with SRS, EI-LSH uses less I/O over all the experiments.

6.10 Summary

Based on the experimental results, we make the following observations:

- Under the same theoretical guarantee, the I/O performance of EI-LSH consistently outperforms QALSH and SRS under all settings. This is because: (1) EI-LSH adopts a natural incremental search strategy, which

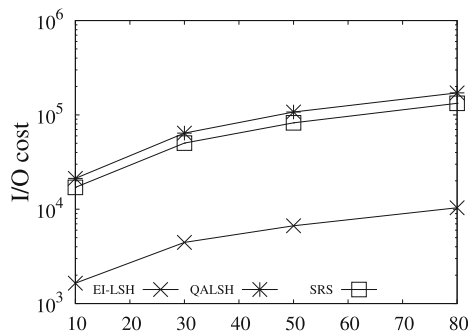


Fig. 13 I/O cost on difference dataset size ($k = 50$)

enables us to achieve c -approximation (unlike the c^2 approximation of QALSH); (2) EI-LSH uses a much more aggressive early termination technique, which can significantly reduce I/O costs without sacrificing the theoretical guarantee; and (3) EI-LSH can take advantage of efficient sequential I/O brought by the B+ tree.

- SRS has a good performance when $c = 4$, but it cannot comfortably support a smaller c (higher accuracy) because of the curse of dimensionality for the exact NN search on the multi-dimensional index including R-tree. Moreover, the random I/O incurred by R-tree is also a limitation of SRS.
- The performance of QALSH is not competitive in relation to I/O efficiency under the same theoretical guarantee. This is mainly because: (1) the c^2 approximate algorithm resulting from its bucket expansion search strategy; and (2) the conservative early termination strategy may miss many opportunities to stop the algorithm without breaking the theoretical guarantee.
- I-LSH and EI-LSH require the same sized index and the same parameter settings to hold a given theoretical guarantee, but EI-LSH adopts a more aggressive early termination condition and can stop earlier than I-LSH.
- The value of P_{ET} has a dramatic effect on the I/O efficiency of EI-LSH when the dataset is enormous. In most cases, our algorithm is stopped by the early termination condition, and on extremely large dataset a smaller P_{ET} can reduce the candidate set size.

7 Conclusion

To deal with the fundamental similarity search on large scale high-dimensional data, in this paper we investigate the problem of c -ANN search and propose an aggressive early termination condition, which can be used on most list-based LSH algorithms to reduce the I/O cost. Furthermore, we also develop an I/O efficient incremental LSH method, namely I-LSH. We show that, by applying an incremental search strategy and the new early termination technique, I-

LSH can significantly enhance I/O efficiency under the same theoretical guarantee. Our comprehensive experiment results show that, compared with state-of-the-art I/O efficient c -ANN techniques, our algorithm can achieve much better I/O efficiency under the same theoretical guarantee. The experiment on the large-scale datasets shows that our algorithm also performs well on non-trivial datasets.

References

1. Arora, A., Sinha, S., Kumar, P., Bhattacharya, A.: Hd-index: pushing the scalability-accuracy boundary for approximate knn search in high-dimensional spaces. *Proce. VLDB Endow.* **11**(8), 906–919 (2018)
2. Bahmani, B., Goel, A., Shinde R.: Efficient distributed locality sensitive hashing. In: *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*, pp. 2174–2178. ACM, New York (2012)
3. Bast, H., Majumdar, D., Schenkel, R., Theobald, M., Weikum, G.: Io-top-k: index-access optimized top-k query processing. In: Dayal, U., Whang, K., Lomet, D.B., Alonso, G., Lohman, G.M., Kersten, M.L., Cha, S.K., Kim, Y. (eds.) *Proceedings of the 32nd International Conference on Very Large Data Bases*, Seoul, Korea, 12–15 September 2006, pp. 475–486. ACM, New York (2006)
4. Bernhardsson, E.: Annoy at github. <https://github.com/spotify/annoy> (2015)
5. Chen, D., Sun, G., Gong, N.Z., Zhong, X.: Efficient top-k query algorithms using density index. In: Zeng, D. (ed.) *Applied Informatics and Communication—International Conference, ICAIC 2011, Xi'an, China, 20–21 August 2011, Proceedings, Part I, Communications in Computer and Information Science*, vol. 224, pp. 38–45. Springer, Berlin (2011)
6. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.S.: Locality-sensitive hashing scheme based on p-stable distributions. In: *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, pp. 253–262. ACM, New York (2004)
7. Deshpande, P.M., Padmanabhan, D., Kummamuru, K.: Efficient online top-k retrieval with arbitrary similarity measures. In: Kemper, A., Valduriez, P., Mouaddib, N., Teubner, J., Bouzeghoub, M., Markl, V., Amsaleg, L., Manolescu, I. (eds.) *Proceedings of EDBT 2008, 11th International Conference on Extending Database Technology*, Nantes, France, 25–29 March 2008, *ACM International Conference Proceeding Series*, vol. 261, pp. 356–367. ACM, New York (2008)
8. Dong, W., Charikar, M., Li, K.: Efficient k-nearest neighbor graph construction for generic similarity measures. In: *WWW* (2011)
9. Fagin, R.: Combining fuzzy information: an overview. *SIGMOD Rec.* **31**(2), 109–118 (2002)
10. Fagin, R., Lotem, A., Naor, M.: Optimal aggregation algorithms for middleware. *J. Comput. Syst. Sci.* **66**(4), 614–656 (2003)
11. Gan, J., Feng, J., Fang, Q., Ng, W.: Locality-sensitive hashing scheme based on dynamic collision counting. In: *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pp. 541–552. ACM, New York (2012)
12. Gao, J., Jagadish, H.V., Lu, W., Ooi, B.C.: DSH: data sensitive hashing for high-dimensional k-nnsearch. In: *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, pp. 1127–1138. ACM, New York (2014)
13. Gao, J., Jagadish, H.V., Ooi, B.C., Wang, S.: Selective hashing: closing the gap between radius search and k-nn search. In: *SIGKDD* (2015)

14. Grover, A., Leskovec, J.: node2vec: scalable feature learning for networks. In: ACM SIGKDD, pp. 855–864 (2016)
15. Gu, Y., Guo, Y., Song, Y., Zhou, X., Yu, G.: Approximate order-sensitive k-nn queries over correlated high-dimensional data. *IEEE Trans. Knowl. Data Eng.* **1**, 1–1 (2018)
16. Haghani, P., Michel, S., Aberer, K.: Distributed similarity search in high dimensions using locality sensitive hashing. In: Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology, pp. 744–755. ACM, New York (2009)
17. Holland, S.M.: Principal components analysis (PCA). Department of Geology, University of Georgia, Athens, GA, pp. 30602–2501 (2008)
18. Huang, Q., Feng, J., Zhang, Y., Fang, Q., Ng, W.: Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proc. VLDB Endow.* **9**(1), 1–12 (2015)
19. Indyk, P., Motwani, R.: Approximate nearest neighbors: towards removing the curse of dimensionality. In: Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, 23–26 May 1998, pp. 604–613 (1998)
20. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.* **33**(1), 117–128 (2011)
21. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with gpus. *CoRR* (2017) [arXiv:1702.08734](https://arxiv.org/abs/1702.08734)
22. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems, pp. 1097–1105 (2012)
23. Kumar, R., Punera, K., Suel, T., Vassilvitskii, S.: Top-*k* aggregation using intersections of ranked inputs. In: Baeza-Yates, R., Boldi, P., Ribeiro-Neto, B.A., Cambazoglou, B.B. (eds.) Proceedings of the Second International Conference on Web Search and Web Data Mining, WSDM 2009, Barcelona, Spain, 9–11 February 2009, pp. 222–231. ACM, New York (2009)
24. Li, W., Zhang, Y., Sun, Y., Wang, W., Zhang, W., Lin, X.: Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement (v1.0). *CoRR* (2016). [arXiv:1610.02455](https://arxiv.org/abs/1610.02455)
25. Liu, W., Wang, H., Zhang, Y., Wang, W., Qin, L.: I-lsh: I/o efficient c-approximate nearest neighbor search in high-dimensional space. In: 2019 IEEE 35th International Conference on Data Engineering (ICDE), pp. 1670–1673. IEEE (2019)
26. Liu, Y., Cheng, H., Cui, J.: PQBF: i/o-efficient approximate nearest neighbor search by product quantization. In: CIKM, pp. 667–676 (2017)
27. Liu, Y., Cui, J., Huang, Z., Li, H., Shen, H.T.: Sk-lsh: an efficient index structure for approximate nearest neighbor search. *Proc. VLDB Endow.* **7**(9), 745–756 (2014)
28. Lv, Q., Josephson, W., Wang, Z., Charikar, M., Li, K.: Multi-probe lsh: efficient indexing for high-dimensional similarity search. In: Proceedings of the 33rd International Conference on Very Large Data Bases, pp. 950–961. VLDB Endowment (2007)
29. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *CoRR* (2016)
30. Muja, M., Lowe, D.G.: Scalable nearest neighbor algorithms for high dimensional data. *IEEE Trans. Pattern Anal. Mach. Intell.* **36**(11), 2227–2240 (2014)
31. Pan, J., Manocha, D.: Bi-level locality sensitive hashing for k-nearest neighbor computation. In: Data Engineering (ICDE), 2012 IEEE 28th International Conference on, pp. 378–389. IEEE (2012)
32. Panigrahy, R.: Entropy based nearest neighbor search in high dimensions. In: Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, 22–26 January 2006, pp. 1186–1195 (2006)
33. Park, Y., Cafarella, M.J., Mozafari, B.: Neighbor-sensitive hashing. *PVLDB* **9**(3), 144–155 (2015)
34. Perozzi, B., Al-Rfou, R., Skiena, S.: Deepwalk: online learning of social representations. In: ACM SIGKDD, pp. 701–710 (2014)
35. Schenkel, R., Broschart, A., Hwang, S., Theobald, M., Weikum, G.: Efficient text proximity search. In: Ziviani, N., Baeza-Yates, R.A. (eds.) String Processing and Information Retrieval, 14th International Symposium, SPIRE 2007, Santiago, Chile, 29–31 October 2007, Proceedings, Lecture Notes in Computer Science, vol. 4726, pp. 287–299. Springer, Berlin (2007)
36. Silpa-Anan, C., Hartley, R.I.: Optimised kd-trees for fast image descriptor matching. In: CVPR (2008)
37. Sun, Y., Wang, W., Qin, J., Zhang, Y., Lin, X.: SRS: solving c-approximate nearest neighbor queries in high dimensional Euclidean space with a tiny index. *Proc. VLDB Endow.* **8**(1), 1–12 (2014)
38. Tao, Y., Yi, K., Sheng, C., Kalnis, P.: Quality and efficiency in high dimensional nearest neighbor search. In: Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data, pp. 563–576. ACM, New York (2009)
39. Theobald, M., Bast, H., Majumdar, D., Schenkel, R., Weikum, G.: Topx: efficient and versatile top-*k* query processing for semistructured data. *VLDB J.* **17**(1), 81–115 (2008)
40. Wang, J., Huang, P., Zhao, H., Zhang, Z., Zhao, B., Lee, D.L.: Billion-scale commodity embedding for e-commerce recommendation in Alibaba. In: ACM SIGKDD, pp. 839–848 (2018)
41. Wang, J., Zhang, T., Song, J., Sebe, N., Shen, H.T.: A survey on learning to hash. *IEEE Trans. Pattern Anal. Mach. Intell.* **40**(4), 769–790 (2018)
42. Wang, Y., Shrivastava, A., Ryu, J.: Flash: randomized algorithms accelerated over CPU-GPU for ultra-high dimensional similarity search (2017). [arXiv preprint arXiv:1709.01190](https://arxiv.org/abs/1709.01190)
43. Weiss, Y., Torralba, A., Fergus, R.: Spectral hashing. In: NIPS, pp. 1753–1760 (2008)
44. Zhang, J., Khoram, S., Li, J.: Efficient large-scale approximate nearest neighbor search on OpenCL FPGA. In: CVPR, pp. 4924–4932 (2018)
45. Zheng, Y., Guo, Q., Tung, A.K., Wu, S.: Lazyish: approximate nearest neighbor search for multiple distance functions with a single index. In: Proceedings of the 2016 International Conference on Management of Data, pp. 2023–2037. ACM, New York (2016)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.